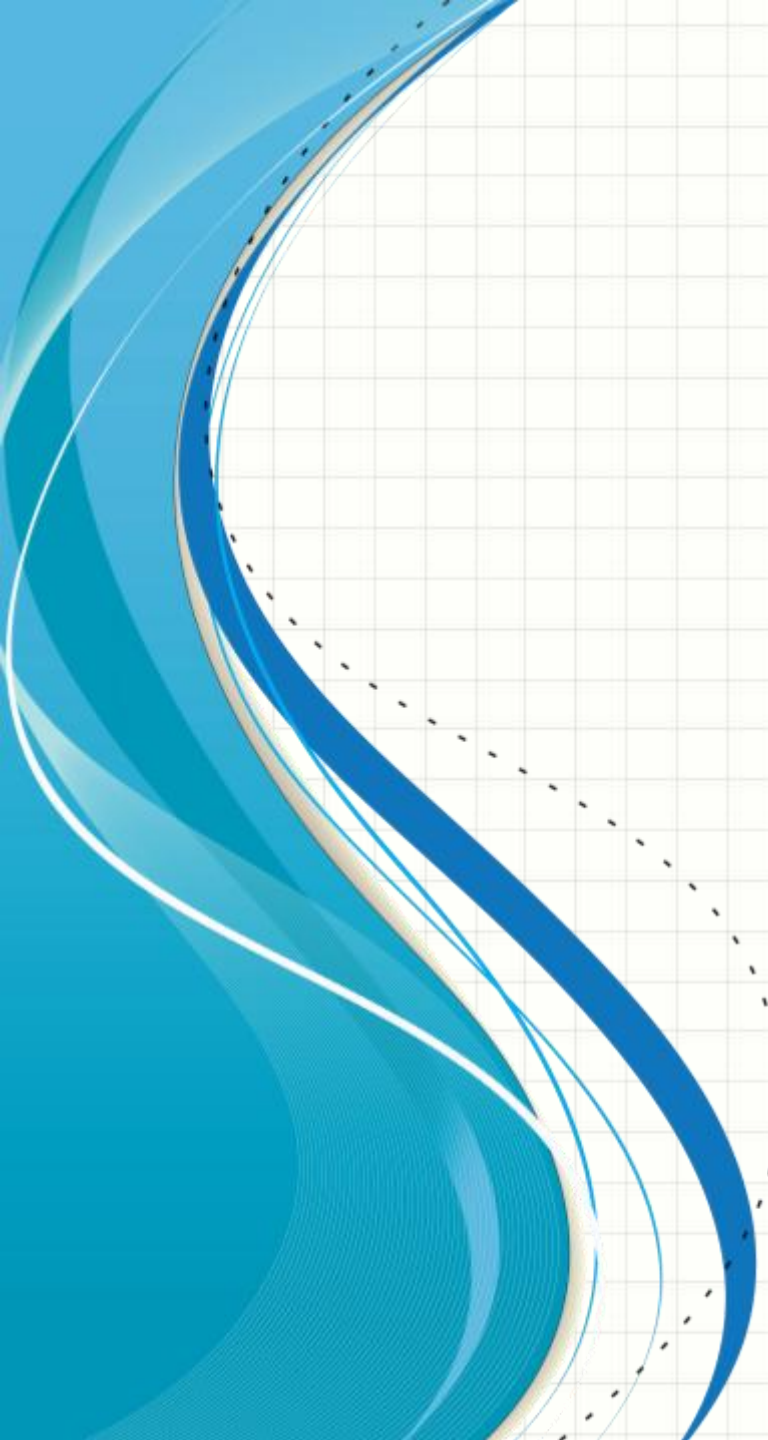




ILLINOIS INSTITUTE
OF TECHNOLOGY

Transforming Lives. Inventing the Future.

www.iit.edu

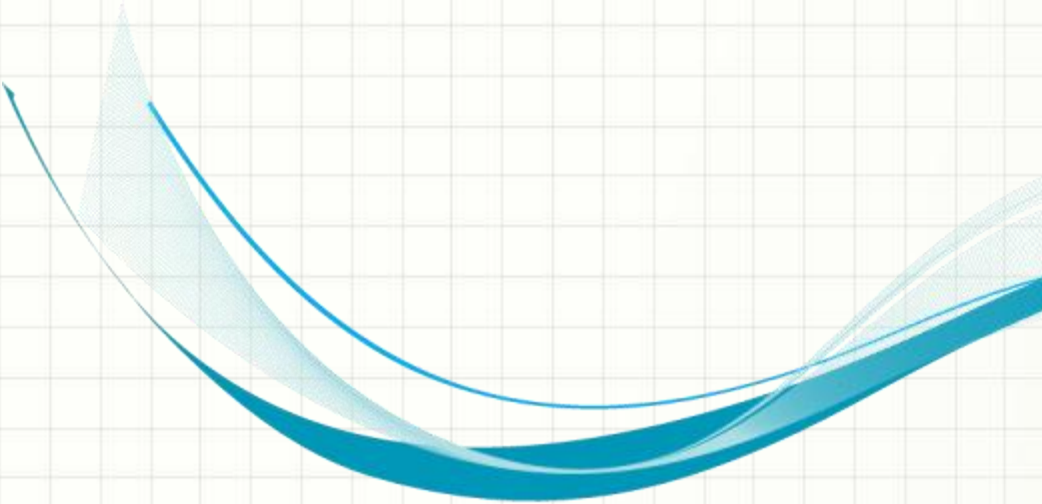
SOFTWARE ENGINEERING

CS 487

Prof. Dennis Hood
Computer Science

Lesson Overview

- Dynamic-Binding Architectures
- Reading
 - Ch. 17 – Distributed Software Engineering
 - Ch. 18 – Service-oriented Software Engineering
- Objectives
 - Systems are collections of cooperating sub-systems
 - As discussed previously in several contexts, the overall success of a system is dependent in large part on the predictable and reliable interaction with the various sub-systems
 - Service-oriented architectures extend to include sub-systems to which the “main” system binds at runtime
 - Agentic AI can manage open-ended pursuit of goals



Week 12

Dynamic-Binding Architectures

Human Service Providing

- Humans play various service-related roles:
 - We take care of our own needs
 - We ask for assistance when needed
 - This can be casual and informal (friends/family), or
 - It can be a formal request with contract parameters
 - We help others when needed
 - Again, informal or formal
- Service providing comes with expectations
 - Specification of parameters
 - Time to complete
 - Correctness and completeness
 - Reliability and consistency

Automating Service Providing

- Who is in control?, Who is providing support?
 - We perceive control to be based in our system
 - Control is temporarily handed to a partner at the request/command of our system
 - Control is returned to our system when the partner has completed its finite processing to satisfy our request
- What could go wrong?, How can it be managed?
 - We can't do it ourselves – call a lifeline
 - The call is not answered – call another one
 - The provider is unavailable – call another one
 - The provider agrees, but fails to deliver – call another one
 - There are no more providers – do it ourselves, or try again later

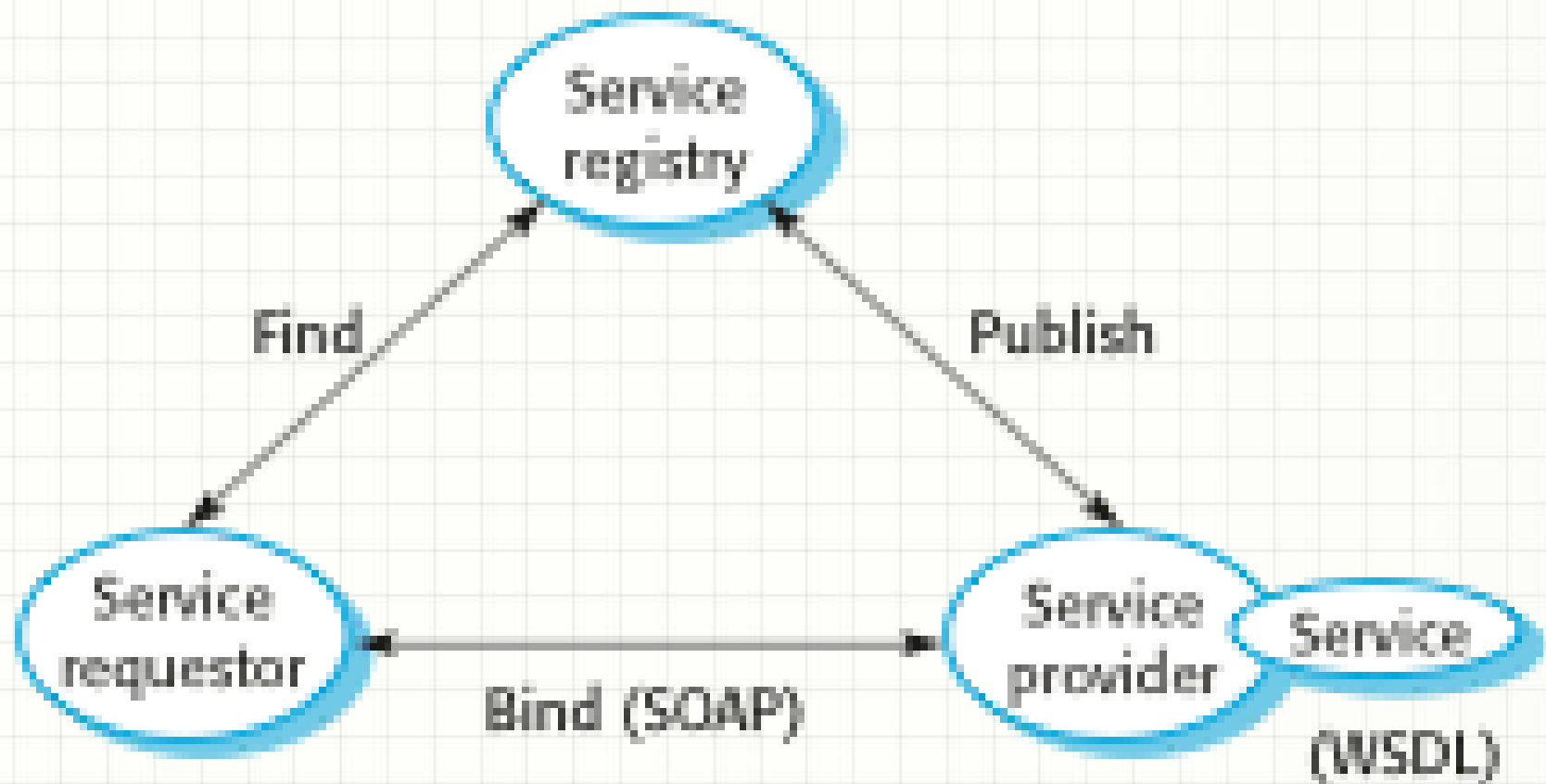
Architecture Alternatives

- Static and internal
 - Within the central system in our context model
 - Functions/methods/routines within our system
 - Designed, implemented, and tested by us, then released to users
- Static and external (traditional partners)
 - The fringe of our context model – specialized partners
 - Established protocols to govern control – tested, then released
 - Reliability concerns because “we didn’t build it”
- Dynamic service providers (SOA)
 - Find potential partners as needed, and request binding
 - Partners should be certified as predictably reliable
 - Manage the associated risks with exception management
- Dynamic agents
 - Utilize a control agent that will in turn find partner agents when needed, and manage the temporary relationships
 - Non-deterministic, and therefore least predictable, knowable, controllable

Service-Oriented Architecture (SOA)

- Based on the notion of cooperating entities
 - Entities offer to perform services
 - Information can be made available in a controlled and managed way
- Defined interfaces must be published
- Allows for the creation of configurable, distributed, platform-independent systems

Service-Oriented Architecture



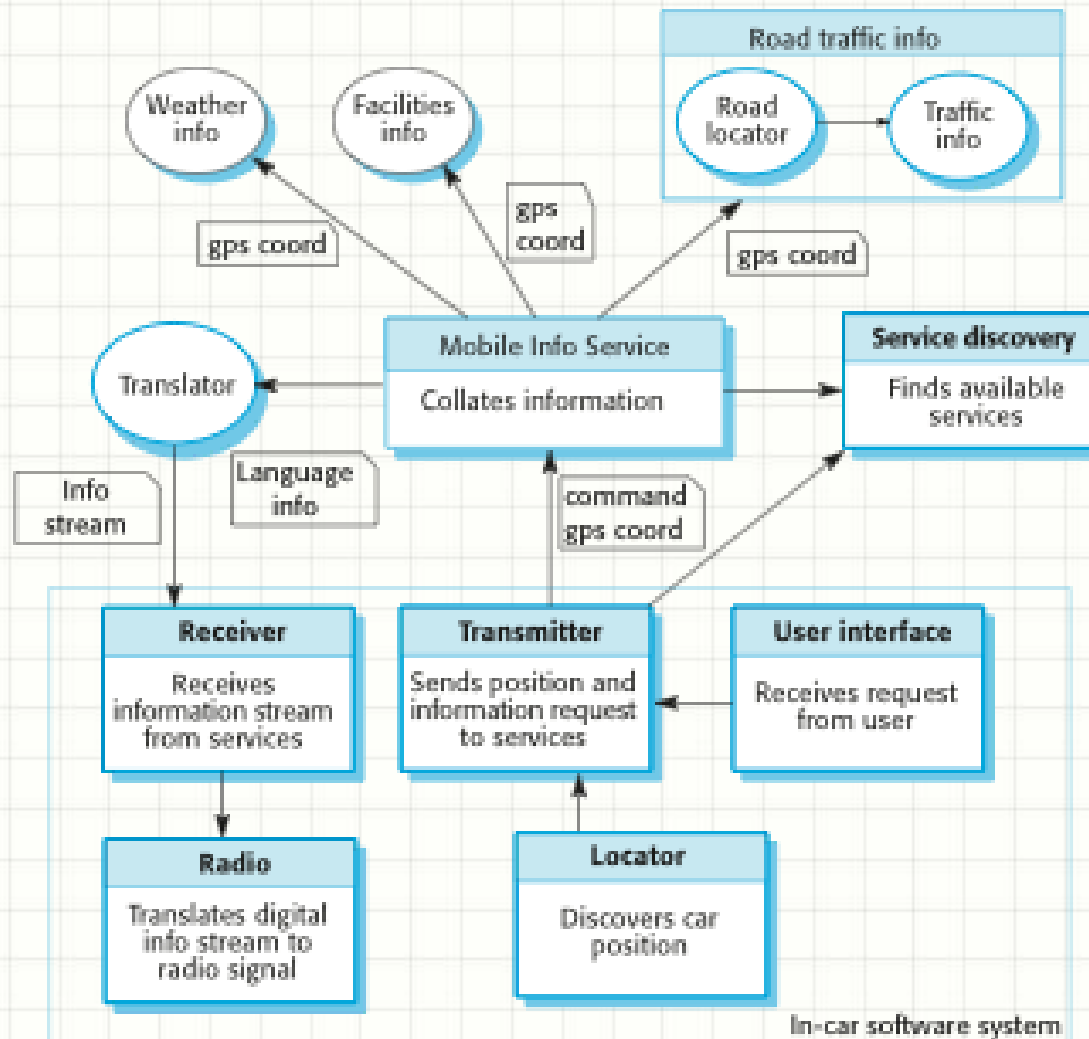
SOA-related Standards

- SOAP – SOA Protocol
 - Message interchange
 - Supports communication between services
- WSDL – Web Services Definition Language
 - Service interface definition
 - Establishes definition for service operations and bindings
- WS-BPEL – Web Services Business Process Execution Language
 - Workflow language

Benefits of SOA

- Services can be provided locally or outsourced to external providers
- Services are language-independent
- Investment in legacy systems can be preserved
- Inter-organizational computing is facilitated through simplified information exchange
- Reuse

Ex.: Car Information System



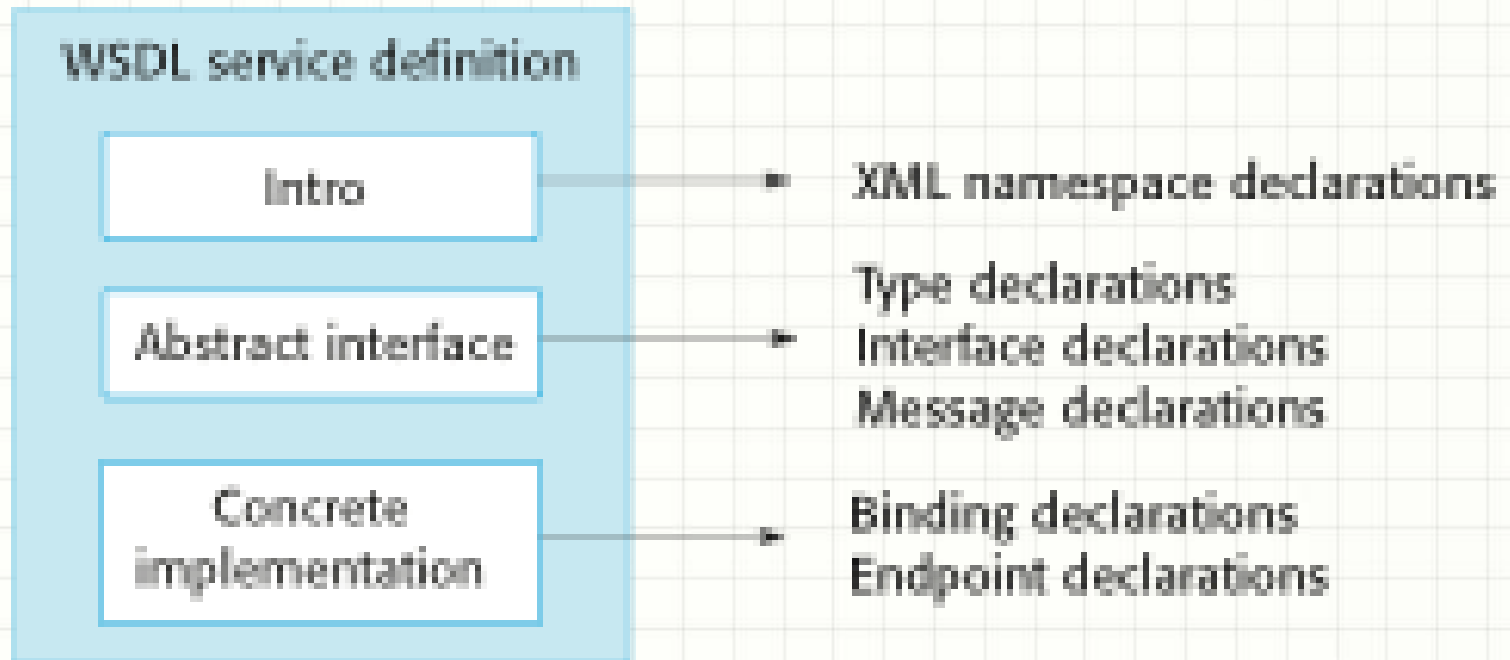
Ex.: Car Info Systems (cont.)

- It is not necessary to decide when the system is programmed or deployed what service provider should be used or what specific services should be accessed
 - As the car moves around, the in-car software uses the service discovery service to find the most appropriate information service and binds to that
 - Because of the use of a translation service, it can move across borders and therefore make local information available to people who don't speak the local language

Reusable Services

- Once established, a service can be used by any number of “systems”
 - Well-defined interface
 - Consistent, reliable performance
 - Independent and loosely coupled
- WSDL specification
 - What – the interface description
 - How – details of how to communicate
 - Where – the location of the implementation

WSDL Specification



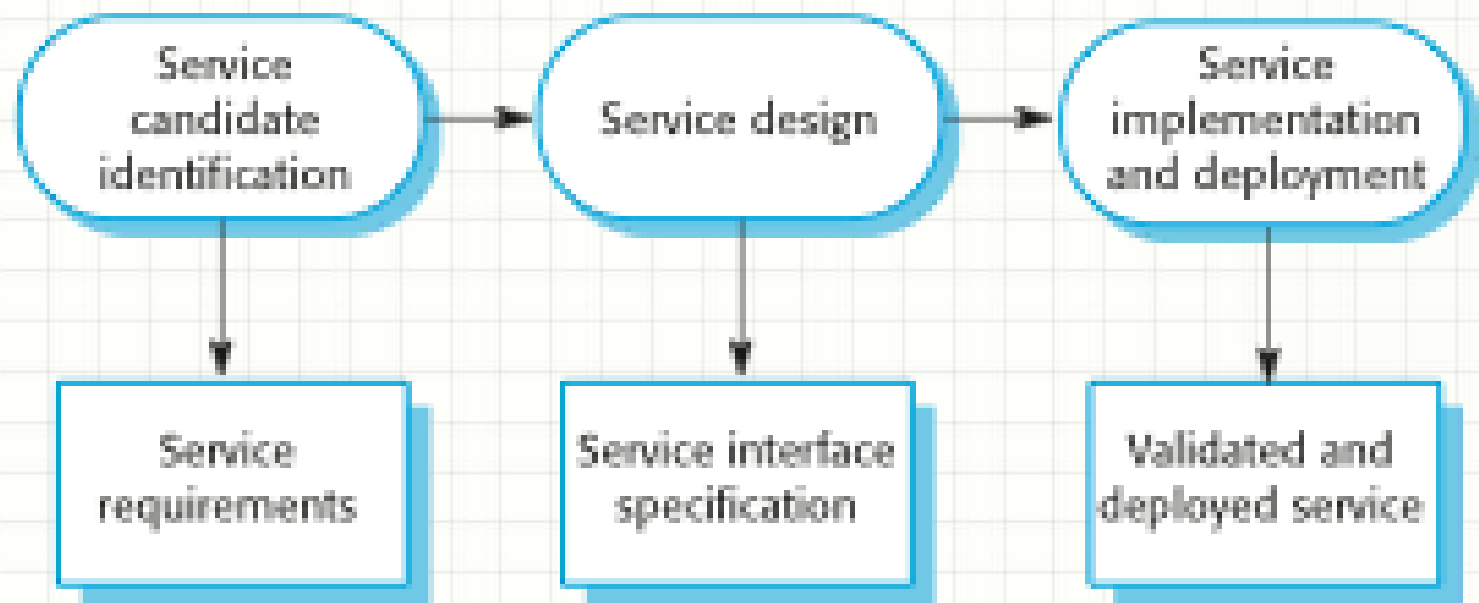
Service Types

- Utility services
 - General functionality
 - e.g., currency conversion
- Business services
 - Associated with a specific business function
 - e.g., student registration
- Coordination of process services
 - Support more general business processes
 - e.g., procurement process management

Service Engineering

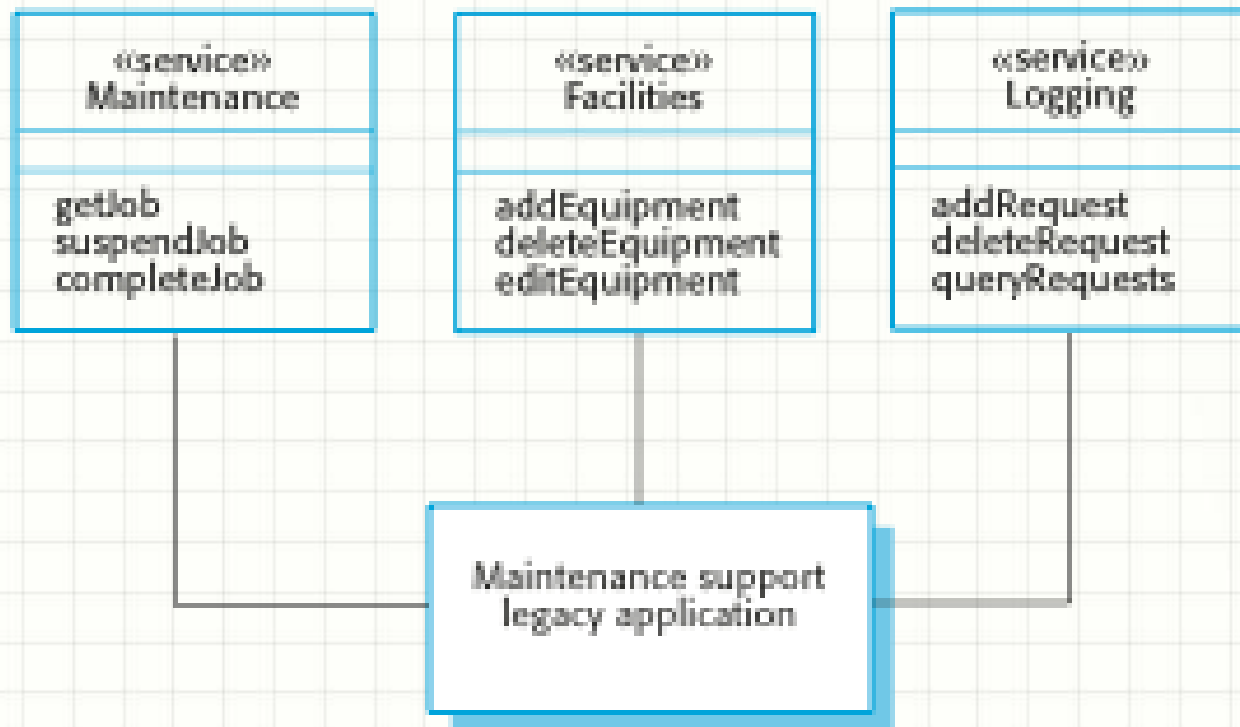
- The process of developing services for reuse in service-oriented applications
- The service has to be designed as a reusable abstraction that can be used in different systems
- Generally useful functionality associated with that abstraction must be designed and the service must be robust and reliable
- The service must be documented so that it can be discovered and understood by potential users

Service Engineering



Legacy System Wrappers

- Extend the life of legacy systems
 - Provide an interface in the form of a wrapper instead of rewriting (or “sun-setting”)



Service Testing Considerations

- Control or even access to external services is determined by the service provider
- An application may not always use the same service each time it is executed
- Performance may differ under varying loads – loads which can vary greatly depending on the number and frequency of service requests
- Testing of exception handling may depend on proper failure of dependent services

Agentic AI

- Autonomous agents
 - Goal-oriented
 - Aware of their environment
 - Able to reason and make decisions
 - (Presumably) able to take the best actions for achieving the stated goals
 - Learn and adapt
 - The future is uncertain

Hybrid Dynamic Architectures

- SOA for the foundation
 - Data management
 - Basic communication and control
- Agentic for the decision-making and learning
 - Understanding current state
 - Predicting needs
 - Finding and binding
 - Assessing success in meeting goals
 - Learning to be better (including potentially new goals)

Managing Risk

- Use the approach that best fits the need
 - Need for predictability, satisfaction of requirements
 - Open-ended, pursuit of goals
 - Tolerance for delays
 - Acceptance of the unknown
- Develop exception management where needed (system-partner interactions)
 - Binding
 - Parameter passing
 - Timeouts
 - Lack of availability
- Proactive and reactive QA
 - Registration based on certification
 - Protocols
 - Test thoroughly